Perch: technical spec

Perch is a native macOS app that turns the MacBook notch into one inbox for AI agents and messages. Claude Code and Codex sessions, Grok Bot agents, Slack and WhatsApp all show up in the same island: hover to open it, click a row to read and reply, and it opens by itself when something needs you (a permission prompt, a question, a new message).

This document describes how the current build works, so it can be rebuilt from scratch. It is a personal tool, not a product: some integrations use private or unofficial interfaces, which are called out where they appear.

1. Stack at a glance

Piece	Technology
App	Swift 5.9, SwiftUI with `@Observable` models, AppKit `NSPanel`, macOS 14+, Swift Package Manager (one executable target)
Claude Code and Codex	Their hook systems, a shell hook script, and an HTTP server on an owner-only unix socket inside the app
Codex (extra)	Watching Codex's rollout files in `~/.codex/sessions`, plus its CLI queue for replies
Grok Bot	A long-lived Node bridge (`bridge/grok-bridge.mjs`) reusing the `grokbot` Claude Code plugin's gateway client (private API)
Slack	A Slack app per workspace in Socket Mode (websocket), with a user token for reading and sending
WhatsApp	A Go helper (`perch-whatsapp`) built on whatsmeow, joined as a linked device, talking JSON lines over stdin/stdout
Search	Fuzzy text search plus on-device meaning search with bge-small-en-v1.5, converted to Core ML (int8)
Notion	Notion API with a token kept in a local file, for a to-do board
Build	`scripts/build-app.sh` : SwiftPM build, Core ML model compile, Go helper build, app bundle assembly, code signing

Roughly 17k lines: about 15k of Swift (most of it SwiftUI views), 1k of Go, plus small shell, Python and Node scripts.

2. The window

The notch UI is one `NSPanel` that never becomes the key window and never resizes.

- **Fixed transparent canvas.** The panel is a large transparent canvas hanging from the top of the screen. Opening and closing the island is a pure SwiftUI animation inside it, so there is no window resizing jank. Mouse events pass through everywhere except over the island itself (hit-testing against the island's current shape).
- **Above full screen apps.** A private window-server call puts the panel in its own top-level space, so it sits above every desktop and full screen space and does not slide when a full screen app reveals its menu bar. This is the same trick other notch apps use. If a macOS update breaks it, the island falls back to behaving like a normal floating panel.

- **Edge guard.** An event tap (needs the Accessibility permission) stops the pointer touching the very top pixel while it is over the island, so full screen apps never get the cue to drop their menu bar. It runs on its own thread so pointer movement never waits on the UI.
- **First click counts.** The panel accepts the first mouse click, so clicking a row works straight away instead of only focusing the window.
- **Never steals typing.** The app never activates itself on launch or restore (relaunches use `open -g`), so a rebuild or a restored side panel cannot interrupt typing in another app.
- **Faint rim.** A thin, low-contrast stroke around the open island so it does not blend into dark wallpapers.

Layout

- **Closed:** the notch shows small faces of the active agents on each side (Grok bots left, coding sessions right) and their working state.
- **Open:** a left rail with search, the pinned lead bot as a tile at the top, then chats ordered by most recent activity (bots, coding sessions, and unread Slack and WhatsApp conversations). The right side shows the selected chat with a composer.
- **Tabs:** Chats and To-dos (the Notion board).
- **Side panels:** any chat can be popped out into a floating side panel. Open panels are saved and restored after a relaunch, restored without scrolling animation and without taking keyboard focus. Double-clicking a panel's header closes it and opens the real chat in the agent's own app.
- **Banners:** when a message arrives for a chat that is not open, a small banner drops in (or the island opens by itself if it was closed). Hovering a banner shows the full message without marking it read; clicking opens the chat; you can reply or react from the banner.
- **Lightbox:** clicking an image enlarges it inside the island with a close button, instead of opening Preview.
- **Send animation:** sent messages rise into place with a spring transition; the composer clears immediately and the message shows before the network confirms it (optimistic send).

3. Data model

- `AgentStore` (`Model.swift`): every agent the notch knows about, ordered by how loudly it should be shown (needs permission > asked a question > finished > working > idle). Each agent has a source (Claude Code, Codex, Grok), a title, current activity, avatar data and unread state.
- `NotchUI` (`NotchWindow.swift`): view state. Open or closed, selected row, reply drafts per chat, banners, search query and results, Slack/WhatsApp unread and dismissed sets.
- `ChatMessage` (`Threads.swift`): one message in a chat, with optional attachments, an options card (a question with choices), an approval card, an email draft card, a secure form card, or a "connect an integration" card.
- `SlackTarget` / `SlackLine` / `SlackFile` (`Slack.swift`): a conversation, one message in it, and a file in a message. WhatsApp reuses these with the workspace name `whatsapp`, so both share the same rail rows, banners, chat view and search.
- Reply drafts, including attached images, are saved to `~/.perch/` as they change, so an unsent draft survives a restart.

All local state lives in `~/.perch/` (folder mode 0700, files 0600).

4. Integrations

4.1 Claude Code and Codex (hooks)

Both tools support hooks: commands they run at points in a turn with the event as JSON on stdin. They share one protocol, so one handler serves both.

- `scripts/install-hooks.py` adds Perch's hooks to Claude Code's and Codex's settings (backing them up first; `--uninstall` reverses it).
- Each hook runs `~/.perch/perch-hook <claude|codex> <mode>`, a small shell script that POSTs the event with `curl --unix-socket ~/.perch/perch.sock`.
- **Fails open:** if Perch is not running, the script exits 0 with no output and the agent behaves exactly as without the hook.
- Inside the app, `HookServer.swift` is a minimal HTTP server on that unix socket. The socket is owner-only, so nothing else on the machine (or a browser) can reach it.
- **Held hooks:** for permission prompts, questions and end of turn, the server keeps the connection open until the user answers in the notch, then writes the answer as the hook's output. That is how "Allow / Deny" in the notch drives the real agent.
- **Replying later:**
  - Within the follow-up window after a turn ends (default one minute), a reply becomes the session's next prompt.
  - A background "wait" hook stays connected after each turn. If a reply is typed while the session is idle, it exits with code 2, which wakes the session with that reply.
  - Otherwise Claude Code is resumed in the background with `claude -p --resume <id>`.
  - Codex replies go through `codex queue --thread <id> --message <text>`, which submits via Codex's own daemon even when the desktop app owns the chat. Failed sends show Try again and survive a restart.
- **Codex rollout watching:** Codex writes every session to a rollout file. Perch tails those incrementally, so Codex sessions (including its desktop app) show even before its hooks are trusted, and Codex's multiple-choice question cards can be answered from the notch.
- **Usage meters:** Claude Code reports plan usage through its status line hook; Codex writes its into each session file. Perch shows the five-hour and weekly bars from those. No account details are read.
- **Skills:** the user's Claude Code skills are indexed so a reply can start with `/name` with autocomplete.
- Perch's own background helper runs set `PERCH_SKIP=1`, so they never appear as chats.

4.2 Grok Bot

Grok Bot (Cursor's agent desktop app) keeps agents on a per-user cloud host. Perch talks to that host the same way the app does.

- **Unofficial:** this uses Grok Bot's private host API, through the gateway client in the `grokbot` Claude Code plugin. An app update can break it.
- `bridge/grok-bridge.mjs` is one long-lived Node process. Perch sends it `{command, body}` requests and reads responses and a live event stream.
- **Commands used:** listing agents (with current activity, avatar shape, last message id, last activity time), loading transcripts, sending messages, marking read/unread, submitting or dismissing secure forms, listing

and connecting listener integrations, and switching an agent's phone notifications off while its chat is open in Perch (and back on after).

- **Fresh-message notifications** are keyed on a change of the agent's last message id, not the unread count, so a bot does not re-notify after it has been read. Messages that arrived just before a relaunch still notify.
- **Activity phrases:** while a bot works, its current tool maps to a short phrase ("Clicking around", "Reading the page"), using the same tool-to-phrase rules as the app.
- **Avatars:** each bot is a 3D ball with painted eyes (`GrokBall.swift`), drawn in a SwiftUI `Canvas` from keyframes measured off a screen recording of the real app. Eyes are projected onto the sphere, so they squash and tilt near the edge. `GrokShapes.swift` holds the 24 avatar outlines (96 points each, plus eye placement), generated from the app's own geometry. A bot without a chosen shape gets the default by hashing its id (FNV-1a) into one of eight.
- **Secure forms:** a bot can ask for details (a sign-in, a check-in) through a form whose answers go straight to the website via Grok Bot, never through the chat. Perch renders the form natively and submits it through the host.
- **Hidden bots:** a list of bots that are searchable but never notify or sit in the notch.

4.3 Slack

- One Slack app per workspace, created from `slack/manifest.json` . It uses Socket Mode (a websocket, so no public server is needed) with user-level scopes to read history, read users and channels, send messages and upload files.
- `slack/add-workspace.sh <name> <priority> [apps|people]` saves the workspace's two keys (the `xapp-` app-level token and the `xoxp-` user token) straight from the clipboard into `~/.perch/slack/<name>.json` (0600), then clears the clipboard. The keys are never typed or shown.
- `apps.connections.open` gives a websocket URL; message events arrive on it live.
- **Heartbeat:** a ping every 30 seconds with a 10 second timeout, and a reconnect on wake from sleep. Without this, the socket can die silently and notifications arrive many minutes late.
- **No stale banners:** messages older than about 90 seconds, or already answered by the user, do not raise a banner.
- **What renders:** real names for @mentions (only "you" when it is actually you), custom emoji (learned per workspace), files with previews and download (file URLs need the workspace token, so Perch fetches them itself), quotes and tweet embeds, full message text.
- **History** loads per conversation, warmed in the background for recent ones, and replaced in place on reload so the view does not jump.
- Unread conversations stay in the rail until read or dismissed.
- Channel rows show the Slack logo as the main icon with the workspace icon small.

4.4 WhatsApp

- **Unofficial:** WhatsApp has no API for personal accounts. Perch joins as a linked device (like WhatsApp Web) using whatsmeow, the Go library most WhatsApp bridges are built on. This is against WhatsApp's terms for automation, so keep it to personal, human-paced use.
- **Helper process:** `whatsapp/main.go` builds `perch-whatsapp` , bundled in the app's Resources. Perch starts it and they exchange one JSON object per line over stdin/stdout.
  - Commands: `status` , `chats` , `history` , `send` , `sendFile` , `markRead` , `download` , `avatar` , `contacts` , `react` , `logout` .

    - Events: `qr`, `paired`, `ready`, `loggedOut`, `synced`, `message`, `reaction`, `status` (delivery and read ticks), `read`, `error`.
- **Linking:** the helper emits QR codes; Perch draws them with `CIQRCodeGenerator` and the user scans from their phone.
- **Storage:** `~/.perch/whatsapp/`: `session.db` (whatsmeow's device keys), `messages.db` (chats and messages, SQLite via `modernc.org/sqlite`, pure Go, no cgo), `media/`, and `helper.log`. The log records event kinds only, never message content.
- **Concurrency lessons** (each was a real freeze):
    - Never do network work (like fetching group info) inside whatsmeow's event handler. Events go onto a queue and a worker goroutine handles them.
    - Open the SQLite databases with more than one connection. A single connection deadlocked receiving against sending.
    - Never run lookups while a `rows` iteration is still open; collect rows first, close, then look up.
- **Identity:** WhatsApp now addresses some people by an anonymous id (LID) instead of their phone number. The helper maps LIDs to phone numbers where known and merges chats that were split across both, so one person is one chat.
- **Names:** contact name if saved, otherwise the name the person set on WhatsApp (push name), otherwise the number.
- **What notifies:** direct chats and small groups (15 people or fewer). Big groups and anything inside a Community only notify when someone @-mentions you or replies to you; the rest stay out of the rail. On connect, the helper reads every joined group in one request (`GetJoinedGroups`) to know each group's size and Community membership up front, instead of guessing when a one-off lookup fails.
- **Status updates** (`status@broadcast`) are ignored entirely.
- **Read state:** nothing is marked read until the chat is actually open in Perch. Hovering a banner does not mark it read. Banners are suppressed while the WhatsApp desktop app is frontmost.
- **UI:** WhatsApp-style green and grey bubbles with time and ticks (clock, one tick, two, blue), photos inside the bubble with the bubble hugging the image, reactions with a hover tray, voice messages that play inline (Opus decoded with a local `ffmpeg`), voice recording, and WhatsApp text formatting (`*bold*`, `_italic_`, `~strike~`, monospace).
- A dismissed chat stays out of the rail until a new message arrives in it.

4.5 Other pieces

- **Notion to-dos:** a board read and written through Notion's API, token in `~/.perch/notion-token`. Cards are fixed size so they do not jump on hover.
- **Clipboard history and file shelf:** recent copies and a shelf to drop files on (file promises and temporary files are copied in so they do not vanish). Copies that password managers mark as secret are never recorded.
- **Link router:** Perch can be the default browser purely to route clicked links into the right Chrome profile, learning per site after asking once.
- **Chrome profiles:** read from Chrome's own `Local State`, so the notch can jump to a profile.
- **Dictation:** Apple's speech recognition, on device where supported, straight into the reply draft.
- **Headphones:** one button to connect Bluetooth headphones and make them the Mac's speaker and mic.

5. Search

One search box across every chat: live and older Claude Code and Codex sessions, Grok bots, Slack and WhatsApp conversations, and people to message.

- **Index:** transcripts and conversations are split into passages and kept in `~/.perch/search-index-<version>.json` (vectors base64-encoded). Unchanged files are skipped by modification time; empty transcripts are remembered as empty.
- **Text matching:** whole words, word starts, small typos (edit distance), simple stemming, phrase and proximity bonuses, and term-frequency weighting. Names (people, chat titles) rank first.
- **Meaning matching:** `Embedder.swift` runs bge-small-en-v1.5, converted to Core ML with int8 weights (about 32 MB on disk), with a WordPiece tokenizer written in Swift. Queries get bge's retrieval prefix. Scores are judged relative to the median for that query, so only clearly related threads show.
    - Run it on the CPU only (`computeUnits = .cpuOnly`). On the Neural Engine the first load took 16 seconds and over 3 GB of memory; on the CPU it is around 20 MB and a few milliseconds per passage.
- **Feel:** debounced 120 ms and run once per pause, so results do not reshuffle on every letter. Snippets are only computed for the top 16 results.
- **Results UI:** grouped into chats containing the words, chats about the same thing, and people to message. Arrow keys and Return work.

6. Build, sign, run

```
./scripts/build-app.sh --run      # build ~/Applications/Perch.app and relaunch it
python3 scripts/install-hooks.py  # install the Claude Code and Codex hooks
```

`build-app.sh`:

1. `swift build -c release`.
2. Compiles the Core ML model package with `xcrun coremlcompiler`.
3. Builds the Go helper with `go build` when `main.go` is newer than the binary.
4. Assembles `Perch.app` (Info.plist, icon, model, helper, bridge) in `~/Applications`.
5. Signs it with a stable self-signed certificate ("Perch Signing") if present, so macOS keeps Accessibility and microphone permissions across rebuilds. Falls back to ad hoc.
6. With `--run`: quits the running copy by its exact path and relaunches with `open -g` (background, never stealing focus).

Prerequisites: Xcode command line tools (Swift 5.9+), Go 1.22+, Node 20+ (for the Grok bridge), `ffmpeg` (WhatsApp voice notes), the `claude` and `codex` CLIs if you use them.

Permissions: Accessibility (edge guard, global shortcuts), Microphone and Speech Recognition (dictation, voice notes), Bluetooth (headphones).

7. Local debug hooks

The hook socket also accepts a few debug commands, for example rendering banner and notification fixtures to a PNG without contacting any service. Standalone regression checks build single files with `swiftc` against small test mains in `tests/`, and `cd whatsapp && go test ./...` covers captions, thumbnails and reactions against a temporary database.

8. Privacy rules the build follows

- Everything stays on the Mac. No Perch server; the only network calls go to the services themselves (Slack, WhatsApp, Notion, the Grok Bot host).
- Secrets live in owner-only files under `~/.perch`, are pasted from the clipboard rather than typed, and are never logged.
- Logs record event types, never message content.
- Hooks fail open, so Perch can never block an agent by not running.

9. Suggested build order for a replica

1. The notch panel: transparent canvas, island animation, click-through, above full screen.
2. Hooks for Claude Code: socket server, hook script, permission prompts held open.
3. Chat view with replies (follow-up window, then `claude -p --resume`).
4. Codex: rollout watching, then hooks and the queue for replies.
5. Slack in Socket Mode, with the heartbeat from day one.
6. Search: text matching first, then the embedding model.
7. WhatsApp helper, following the concurrency lessons in 4.4.
8. Grok Bot, if you use it (accepting that it rides on a private API).